CODE REVIEW REPORT

โปรเจกต์: helpdesk-systemv2 | วันที่รีวิว: 16 พฤษภาคม 2569 | ขอบเขต: Backend Go/Fiber/sqlx, Frontend React/Vite/MUI, SQL dump | เงื่อนไข: วิเคราะห์เท่านั้น ไม่แก้ source code

4	5	5	4
Critical	High	Medium	Low / Later

ภาพรวมสั้น

โปรเจกต์มีทิศทางที่ดีตรงที่ backend แยก feature เป็น handler/service/repository และ frontend มี route ตาม role ชัดเจน แต่ความเสี่ยงหลักตอนนี้ไม่ใช่เรื่อง style; เป็นเรื่อง security, permission, schema/query mismatch และ runtime hygiene ฝั่ง frontend. ควรปิด Critical/High ก่อนทำ refactor สวยงาม เพราะหลายข้อกระทบความถูกต้องและความปลอดภัยโดยตรง.

> **จุดที่ดี**
>
> มีการใช้ bcrypt สำหรับ password, query ส่วนใหญ่ใช้ placeholder `?` จึงลด SQL injection พื้นฐาน, ใช้ feature-based folder ทำให้หา domain เช่น ticket/device/user ได้ไม่ยาก, มี JWT middleware หลัง login และ frontend มีการแยก route ตาม role ซึ่งเป็นพื้นฐานที่ต่อยอดได้.

ลำดับแนะนำ

ลำดับ	ควรทำ	เหตุผล
1	Rotate secrets และย้าย config ออกจาก source	ลดความเสี่ยง credential/token ปลอมทันที
2	ปิด authorization ทุก endpoint สำคัญ	กัน user role ต่ำเรียก API admin โดยตรง
3	ทำ schema canonical + migration	แก้ runtime error จาก column/table ไม่ตรงกัน
4	ลบ password hash จาก response DTO	ลด data exposure
5	แก้ frontend syntax/runtime และ lint config	ทำให้ CI/quality gate ใช้ได้จริง

Findings แยก Severity

Critical

1. JWT secret, DB DSN และ SMTP credential ถูก hard-code ใน source

ปัญหาคืออะไร: ค่า secret และ credential อยู่ใน repository โดยตรง เช่น MySQL DSN ใน backend/main.go:79, JWT secret ใน backend/main.go:100-101 และ backend/internal/auth/login/service/login_service.go:70, SMTP user/password ใน backend/internal/gomail/service/gomail_service.go:45-46

ทำไมควรสนใจ: ถ้า repository หลุดหรือ deploy หลาย environment จะ rotate secret ยาก และผู้ไม่หวังดีอาจปลอม token หรือใช้ SMTP account ได้

แนวทางแก้ / best practice: ย้ายไป environment variables หรือ secret manager, แยก config package, rotate JWT secret และ SMTP app password ทันที, เพิ่ม .env.example ที่ไม่มีค่าจริง

ควรแก้ตอนนี้ไหม: ควรแก้ทันที

Critical

2. API ใช้ authentication แบบกว้าง แต่ authorization ยังแทบไม่มี

ปัญหาคืออะไร: หลัง /login มี jwt middleware ครอบ route ส่วนใหญ่ แต่ role/permission check ใช้เฉพาะ /admin/ping ที่ backend/main.go:150 เท่านั้น route สำคัญ เช่น /users, /roles, /status, /tickets/status, assign technician, delete users เปิดให้ผู้ถือ token ใดก็เรียกได้

ทำไมควรสนใจ: ผู้ใช้ role ต่ำอาจเรียก API admin โดยตรงผ่าน Postman/browser devtools ได้ ถึง frontend จะซ่อนเมนูก็ไม่พอ

แนวทางแก้ / best practice: ทำ middleware เช่น RequireRole(admin), RequireAnyRole(manager,admin), RequireTicketOwnerOrStaff แล้วผูกทุก route ตาม capability; อย่าเชื่อ role guard ฝั่ง React อย่างเดียว

ควรแก้ตอนนี้ไหม: ควรแก้ทันที

Critical

3. response ของ user มี password_hash ติดออก API

ปัญหาคืออะไร: domain.User, UserResponse, UserTechnicianRolesResponse ใส่ json:"password" ให้ PasswordHash ใน backend/internal/domain/user.go:18-27, 46-55, 71-80 และ login handler ส่ง user กลับทั้ง object ที่ backend/internal/auth/login/handler/login.go:32-35

ทำไมควรสนใจ: แม้เป็น hash ก็ไม่ควรส่งไป client เพราะเพิ่มผลกระทบหาก token/localStorage/browser ถูกอ่าน และทำให้ข้อมูลลับกระจาย

แนวทางแก้ / best practice: แยก DTO สำหรับ response เช่น UserPublicResponse ที่ไม่มี PasswordHash/TokenHash; ใช้ json:"-" กับ field sensitive ใน domain/db model

ควรแก้ตอนนี้ไหม: ควรแก้ทันที

Critical

4. schema dump กับ query ปัจจุบันไม่ตรงกันหลายจุด

ปัญหาคืออะไร: SQL dump ใช้ tickets.ticket_id, issue_categories_id, instance_id, status table, departments.dep_id/dep_name ที่ dump-helpdesk_system-202604180937v2.sql:315-344 แต่โค้ดหลายจุดใช้ id, issue_category_id, device_instance_id, ticket_statuses, departments.name เช่น backend/internal/case/repository/ticket_query_db.go:101-110, 214-222, 250-263 และ backend/internal/status/repository/status_db.go:16-39

ทำไมควรสนใจ: ระบบอาจ compile ผ่านแต่ runtime query จะ fail ทันทีเมื่อใช้ database ตาม dump หรือ migrate ไม่ครบ ทำให้ production/debug ยากมาก

แนวทางแก้ / best practice: เลือก schema canonical ชุดเดียว, สร้าง migration versioned, ปรับ query/domain/db tags ให้ตรงกัน และเพิ่ม integration test กับ DB test container หรือ schema fixture

ควรแก้ตอนนี้ไหม: ควรแก้ทันที

High

5. register/set-password flow ยังไม่แข็งแรงและอาจ panic

ปัญหาคืออะไร: SetPassword dereference *req.TokenHash โดยไม่ validate ที่ backend/internal/auth/register/handler/register.go:53; token invitation ไม่มี expiry/used_at ใน repository check ที่ backend/internal/auth/register/repository/register_db.go:95-107; handler return nil หลัง set password ที่ line 57 ทำให้ response ไม่ชัด

ทำไมควรสนใจ: request ที่ขาด token อาจ panic, token ใช้ซ้ำได้, audit ยาก และ UX/client handling ไม่แน่นอน

แนวทางแก้ / best practice: validate input ก่อน dereference, เพิ่ม expires_at/used_at, transaction ตอน set password + mark used, response JSON ชัดเจน

ควรแก้ตอนนี้ไหม: ควรแก้เร็ว

High

6. error handling ส่ง internal error กลับ client และใช้ status code ไม่ตรง

ปัญหาคืออะไร: หลาย handler คืน 500 สำหรับ BodyParser/login fail เช่น backend/internal/auth/login/handler/login.go:21-29, register handler 32-40 และหลาย ticket handler คืน err.Error() ตรง ๆ

ทำไมควรสนใจ: client ได้รายละเอียดภายใน DB/ระบบ, user/password ผิดควรเป็น 401 ไม่ใช่ 500, ทำให้ monitoring แยก bug กับ user error ไม่ได้

แนวทางแก้ / best practice: ทำ error model กลาง เช่น AppError{Code,Status,Message}; map sql.ErrNoRows เป็น 404/401, validation เป็น 400, auth เป็น 401/403; log detail ฝั่ง server เท่านั้น

ควรแก้ตอนนี้ไหม: ควรแก้เร็ว

High

7. main.go ทำหน้าที่มากเกินไปและ composition กระจุกตัว

ปัญหาคืออะไร: backend/main.go:71-220 สร้าง dependency, config, middleware, background job, และ register route ทั้งหมดในไฟล์เดียว ขณะที่ wire/routes มีใช้จริงน้อยมากและ /logina ซ้ำกับ /login

ทำไมควรสนใจ: แก้ feature หนึ่งต้องแตะ main.go ใหญ่ เสี่ยง conflict และยากต่อ test/replace dependency

แนวทางแก้ / best practice: แยก config, db, router modules; ให้ routes.Register รวม route เป็นกลุ่ม auth/tickets/admin/devices; ใช้ wire/deps ให้ครบหรือเอาออกถ้ายังไม่ใช้

ควรแก้ตอนนี้ไหม: ควรทยอยแก้หลังปิด critical

High

8. frontend build/lint hygiene ยังไม่ผ่าน

ปัญหาคืออะไร: npm run lint เจอ 353 errors และ 39 warnings; ตัวอย่าง syntax error ที่ frontend/src/logic/Checktimerepair.jsx:5-6, undefined first_name/last_name ใน frontend/src/store/userSlice.js:43-44, process is not defined ใน Vite หลายไฟล์

ทำไมควรสนใจ: lint fail ทำให้ CI ใช้เป็น quality gate ไม่ได้ และบาง error เป็น runtime bug จริง

แนวทางแก้ / best practice: ตั้งค่า ESLint env ให้เหมาะกับ Vite, ใช้ import.meta.env แทน process.env, แก้ syntax/undefined ก่อน แล้วค่อยลด unused/prop-types เป็นรอบถัดไป

ควรแก้ตอนนี้ไหม: ควรแก้เร็ว โดยเริ่มจาก syntax/runtime

High 9. token ถูกเก็บใน localStorage

ปัญหาคืออะไร: frontend/src/components/LoginPage.jsx:57 และ frontend/src/store/userSlice.js:23 เก็บ token/user ใน localStorage

ทำไมควรสนใจ: localStorage ถูกอ่านได้จาก JavaScript หากมี XSS และ token อายุ 72 ชั่วโมงจาก login_service.go:65 ทำให้ window of compromise ยาว

แนวทางแก้ / best practice: ใช้ httpOnly secure sameSite cookie หรืออย่างน้อยลดอายุ token, refresh-token strategy, CSP และ clear token เมื่อ logout/401

ควรแก้ตอนนี้ไหม: ควรแก้เร็วถ้าจะใช้งานจริง

Medium 10. repository/service/handler separation มีพื้นฐานดี แต่ boundary ยังรั่ว

ปัญหาคืออะไร: มี folder handler/service/repository แยกตาม feature ซึ่งดี แต่ service หลายจุดเป็น pass-through, domain model ผสม db/json, handler รู้อะไรเยอะและ response format ไม่สม่ำเสมอ

ทำไมควรสนใจ: เมื่อระบบโต business rule จะกระจายและเปลี่ยน schema แล้วกระทบ API response

แนวทางแก้ / best practice: แยก db row, domain entity, request/response DTO; service รับผิดชอบ business rule; handler ทำ parse/validate/response เท่านั้น

ควรแก้ตอนนี้ไหม: ทยอยแก้เมื่อแตะ feature นั้น

Medium 11. SQL query มีทั้ง SELECT * และ query string ยาวใน code

ปัญหาคืออะไร: เช่น SELECT * ที่ backend/internal/case/repository/ticket_query_db.go:23,33,170,180 และ backend/internal/case/repository/ticket_db.go:56; query detail ยาวมากใน ticket_query_db.go:382 เป็นต้นไป

ทำไมควรสนใจ: SELECT * ผูกกับ schema โดยไม่รู้ตัวและดึง column เกินจำเป็น ส่วน query ยาวใน code ทำให้อ่าน/test ยาก

แนวทางแก้ / best practice: ระบุ column เสมอ, แยก query ซับซ้อนเป็น named const หรือ query builder ที่มี test, เพิ่ม pagination กับ list endpoint

ควรแก้ตอนนี้ไหม: ทยอยแก้ โดยเริ่มจาก endpoint ที่ใช้บ่อย

Medium 12. ไม่มี transaction ใน operation ที่ควร atomic

ปัญหาคืออะไร: register flow create user -> send email -> save token แยกหลาย step ใน backend/internal/auth/register/service/register_service.go:48-62 และ update ticket status หลาย workflow ไม่มี guard state transition

ทำไมควรสนใจ: เกิด partial state ได้ เช่นสร้าง user แล้วส่งเมลล้ม หรือ token save ล้ม, ticket ถูกเปลี่ยนสถานะผิดลำดับ

แนวทางแก้ / best practice: ใช้ transaction สำหรับ DB changes ที่ต้องไปด้วยกัน, ออกแบบ state machine ของ ticket status และตรวจ current status ก่อน update

ควรแก้ตอนนี้ไหม: ควรแก้ใน flow สำคัญ

Medium

13. background job auto-close อยู่ใน main process แบบง่ายเกินไป

ปัญหาคืออะไร: goroutine ใน backend/main.go:111-119 loop ทุกชั่วโมง ไม่มี context cancellation, locking, observability หรือกันหลาย instance รันซ้ำ

ทำไมควรสนใจ: ถ้า deploy หลาย replica อาจ update ซ้ำ และตอน shutdown คุมไม่ได้

แนวทางแก้ / best practice: ใช้ scheduler/worker แยก, context-aware loop, distributed lock หรือ idempotent query, log metric จำนวน ticket ที่ close

ควรแก้ตอนนี้ไหม: ไว้หลัง critical ถ้ายัง deploy instance เดียว

Medium

14. naming convention ไม่สม่ำเสมอ ไทย/อังกฤษ/old/test ปนกัน

ปัญหาคืออะไร: มี Userold, Usertest, ticket/tikcet typo, dep_id vs department_id, issue_categories_id vs issue_category_id, folder case ที่เป็น keyword-like domain, frontend Manager/manager/tech ปนกัน

ทำไมควรสนใจ: อ่านยากและเพิ่มโอกาส mapping ผิด โดยเฉพาะตอน DB migration

แนวทางแก้ / best practice: กำหนด glossary: ticket, issue_category, device_instance, department; ลบ old/commented code เมื่อมั่นใจ; ใช้ชื่อเดียวกันทุก layer

ควรแก้ตอนนี้ไหม: ทยอยแก้ ไม่จำเป็นต้อง big bang

Low

15. commented-out code และ debug prints เยอะ

ปัญหาคืออะไร: มี fmt.Println ใน main.go:74-75, login service:47, register service และ ticket service หลายจุด รวมถึง block code เก่าในหลาย repository

ทำไมควรสนใจ: noise เยอะ ทำให้อ่านยากและ log production ปนข้อมูลส่วนตัว เช่น email/password debug

แนวทางแก้ / best practice: ใช้ structured logger, log level, ลบ commented-out code ด้วย git history เป็นแหล่งย้อนดู

ควรแก้ตอนนี้ไหม: แก้ทีหลังได้ แต่ควรเก็บเมื่อทำ feature

Low

16. frontend API endpoint กระจายและ hard-code localhost หลายไฟล์

ปัญหาคืออะไร: rg พบ http://localhost:5011 จำนวนมาก เช่น LoginPage.jsx:35, Profile.jsx, Sidebar ต่าง ๆ และ AuthService ยังชี้ http://localhost:5000/api/login

ทำไมควรสนใจ: เปลี่ยน base URL/route ทีต้องไล่แก้หลายจุด และทำให้ environment dev/staging/prod ปน

แนวทางแก้ / best practice: สร้าง axios client กลางพร้อม baseURL/interceptor auth/401, ใช้ VITE_API_URL

ควรแก้ตอนนี้ไหม: ทำหลัง runtime/security

Low

17. ยังไม่มี automated test ใน repo

ปัญหาคืออะไร: go test ./... ผ่าน compile แต่ทุก package เป็น [no test files]; frontend ไม่มี test application code

ทำไมควรสนใจ: การ refactor schema/security จะเสี่ยง regression สูง

แนวทางแก้ / best practice: เริ่มจาก service unit tests, repository integration tests กับ schema จริง, frontend smoke tests สำหรับ login/route guard

ควรแก้ตอนนี้ไหม: เริ่มเพิ่มพร้อมงานแก้ critical/high

Low

18. สิ่งที่ยังไม่จำเป็นต้องแก้ทันที: เปลี่ยน framework หรือ rewrite ใหญ่

ปัญหาคืออะไร: โครงสร้าง feature-based มีทิศทางดีอยู่แล้ว และ Go + Fiber + sqlx / React + MUI ใช้ต่อได้

ทำไมควรสนใจ: rewrite จะกินเวลาและเสี่ยงกว่าการปิดช่อง security/schema/runtime ก่อน

แนวทางแก้ / best practice: รักษา stack เดิมไว้ก่อน ปรับทีละ layer และเพิ่ม test เป็นราวกันตก

ควรแก้ตอนนี้ไหม: ไว้ทีหลัง

Architecture / Layering

โครงสร้างปัจจุบันเป็น feature-based มากกว่า layer-based ล้วน เช่น `internal/case`, `internal/device`, `internal/user` แล้วข้างในแยก handler/service/repository ซึ่งเหมาะกับระบบ business app ขนาดนี้. สิ่งที่ควรปรับคือ boundary: handler ควรรับ/validate/request-response, service ควรถือ business rule, repository ควรถือ SQL เท่านั้น และ domain/db/json model ไม่ควรเป็น struct เดียวกันเมื่อข้อมูล sensitive หรือ schema ไม่ตรง API.

สิ่งที่ยังไม่ต้องแก้ตอนนี้

ยังไม่จำเป็นต้อง rewrite framework, เปลี่ยน database library, หรือ refactor UI ทั้งหมด. ควรหลีกเลี่ยง big-bang rename ทุกไฟล์ในรอบแรก เพราะ schema/security ยังเป็นฐานที่สำคัญกว่า. งาน style เช่นลบ unused imports, rename old/test type, หรือรวม API client ทำได้แบบทยอยหลังระบบ critical path เริ่มนิ่ง.

Verification ที่ทำ

- `go test ./...` ใน backend: ผ่านการ compile ทุก package แต่ไม่มี test files
- `npm run lint` ใน frontend: ล้ม พบ 353 errors และ 39 warnings
- ไม่ได้แก้ source code, ไม่ refactor, ไม่ format ไฟล์ใด ๆ

รายงานนี้สร้างจากการอ่าน source และ SQL dump ใน workspace ณ เวลารันรีวิว โดยไม่ได้แก้ไขโค้ดจริง